Compiladores e Intérpretes

Repaso – Semantico II

Traducciones Avanzadas y Manejo de Información Estrutural

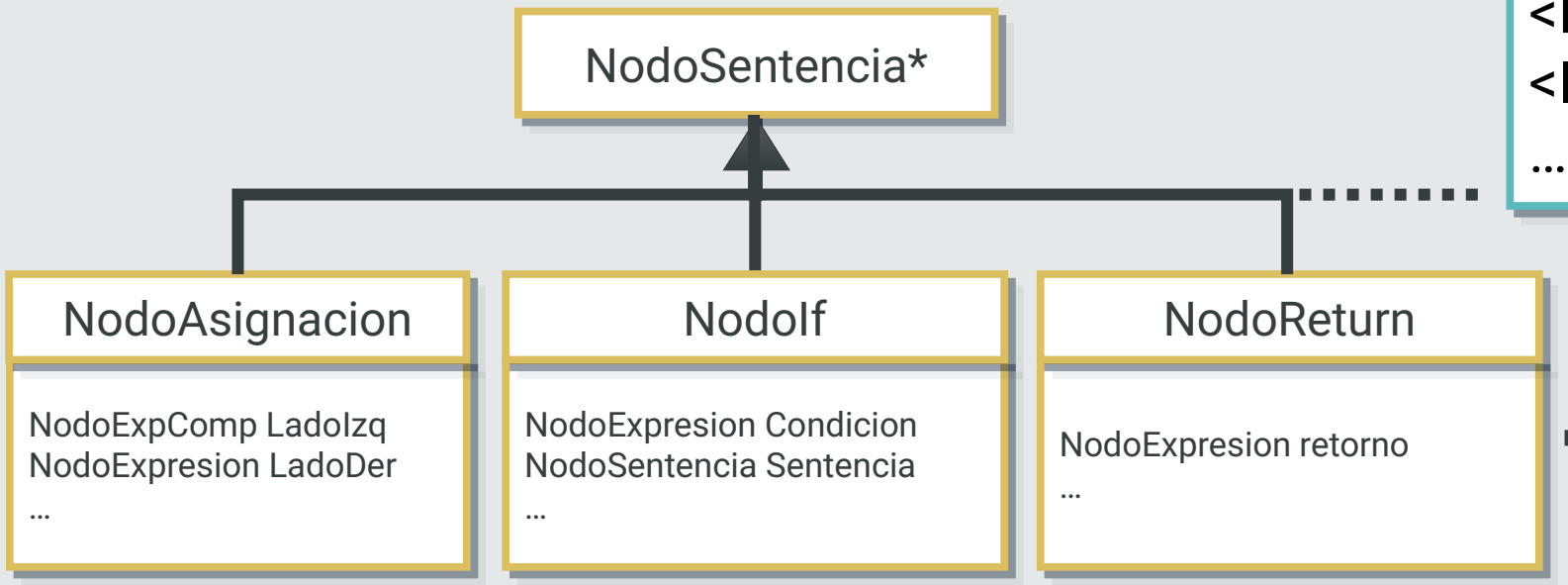
Universidad Nacional del Sur

Departamento de Ciencias e Ingeniería de la Computación

2026

AST – Diseño y Traducción

- En general…



<Sentencia> → <Asignación>;
<Sentencia> → <Return>;
<Sentencia> → <If>

<Return> → **return** <ExpresionOVacio>
<If> → **if**(<Expresion>) <Sentencia>
…

Gramática **MiniJava**

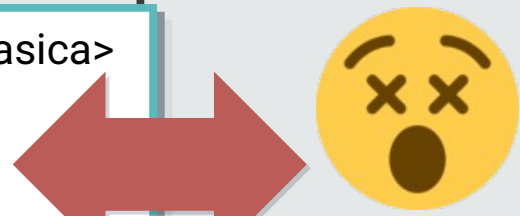
Ayuda a Diseñar… La traducción en el Sintáctico es directa

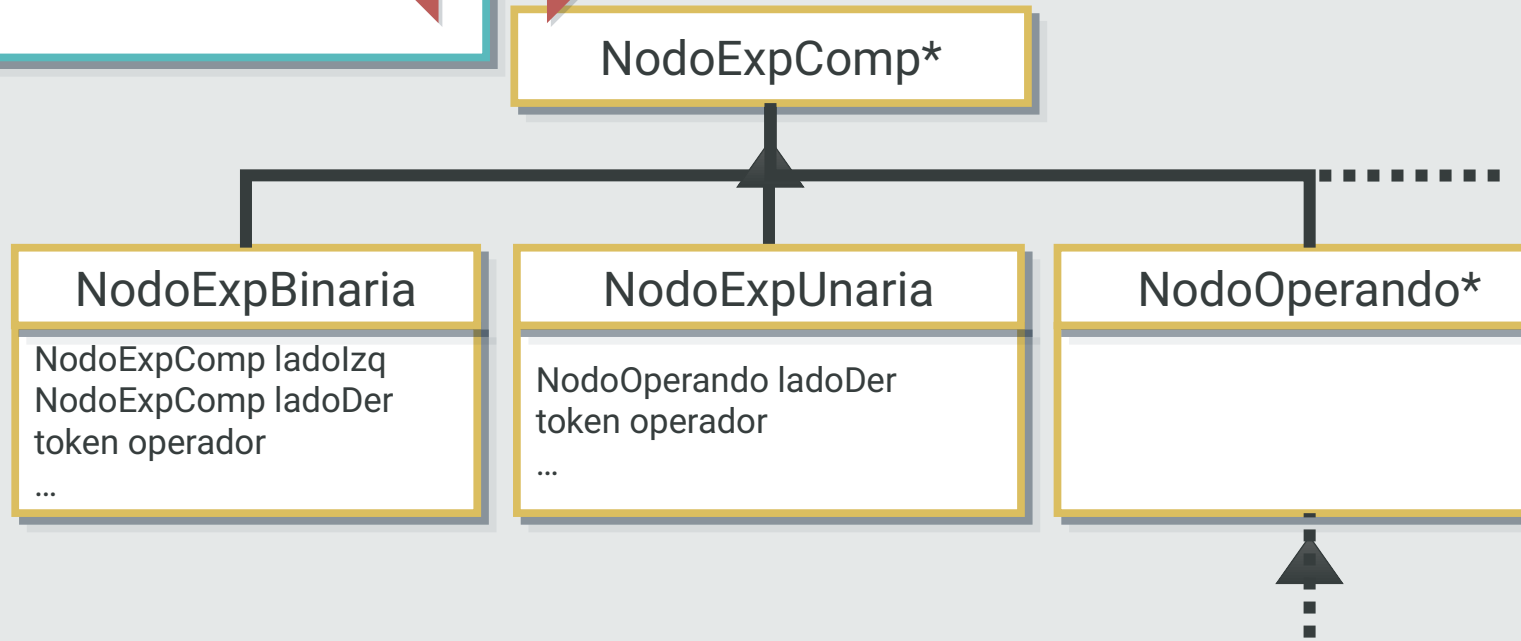
AST – Diseño y Traducción

- Peor aun cuando hay Recursion a Izquierda!

```
<ExpresionComp>→<ExpresionComp> <OperadorBinario> <ExpresionBasica>
<ExpresionComp>→<ExpresionBasica>
<ExpresionBasica>→ <OperadorUnario><Operando>
<ExpresionBasica>→ <Operando>
...
```

Gramática **original MiniJava**

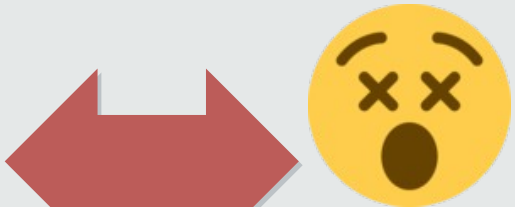


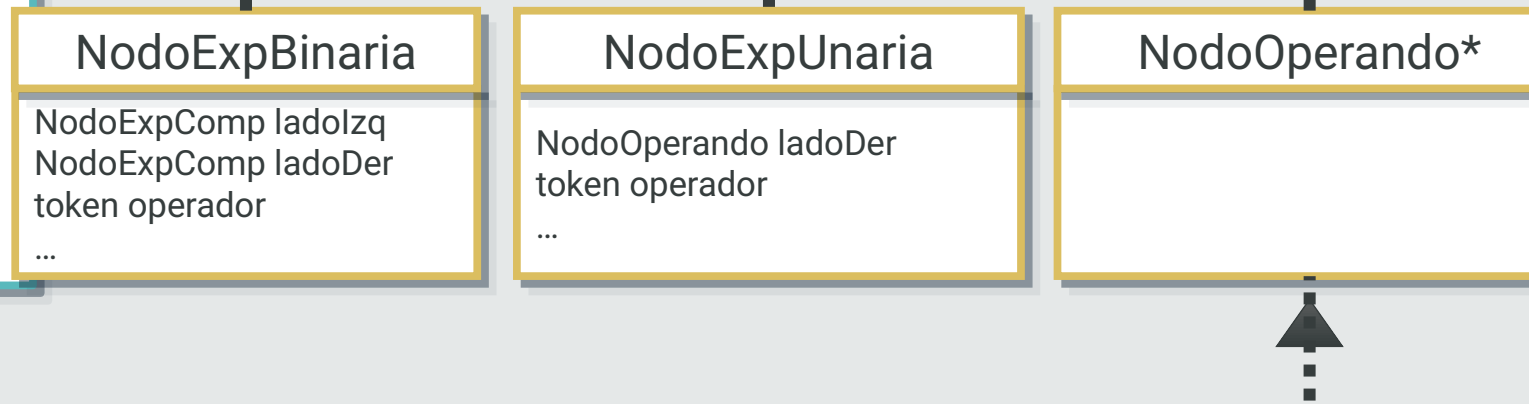


AST – Diseño y Traducción

- Aun que no siempre…

Exp → Operando Resto
Resto → OpBin Operando Resto
Resto → ε
OpBin → **+**
OpBin → ***
Operando → **litInt**
Operando → **id**
Operando → (Exp)





Gramática **Reparada MiniJava**

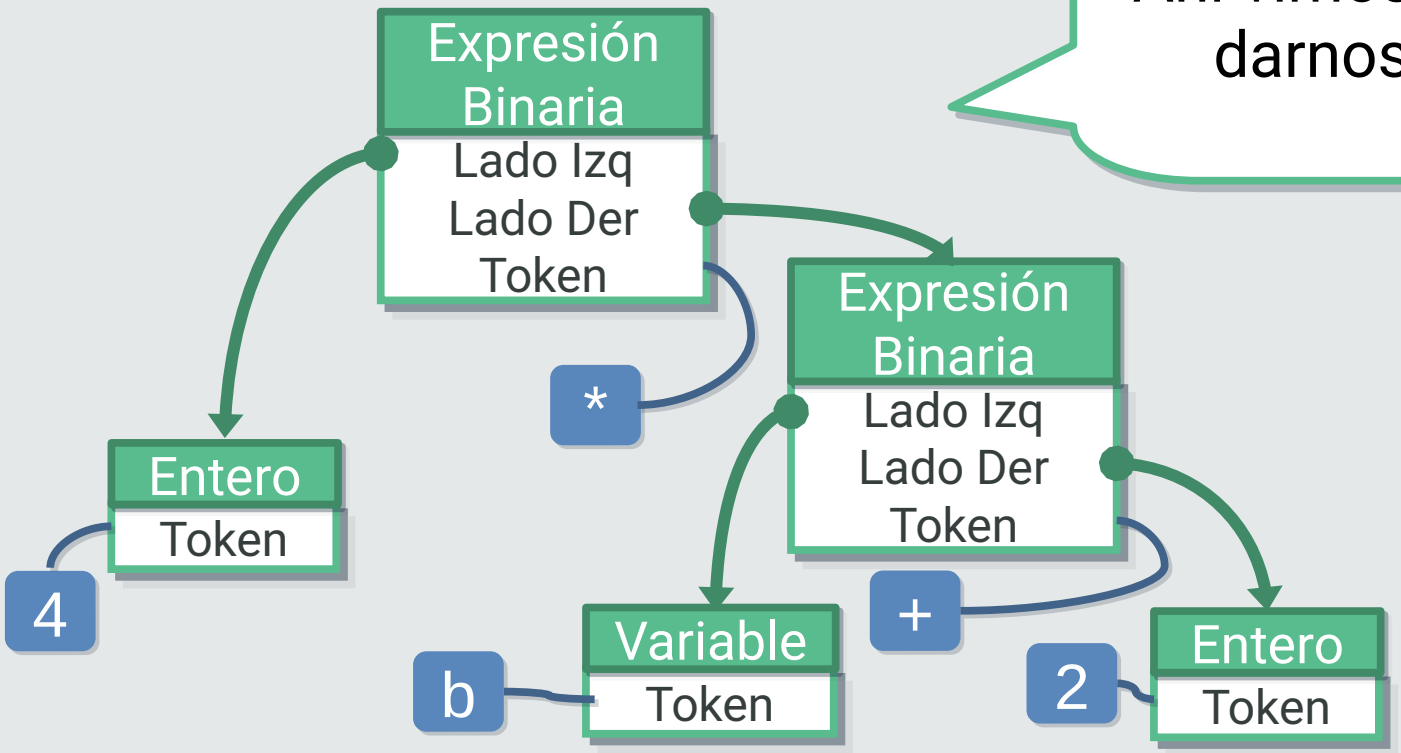
Traduciendo Recursion a Izquierda

```
...
4*(b+2)
...
```

Retomemos el ejemplo de la expresión que vimos el tema 4.5.1

Traduciendo Recursion a Izquierda

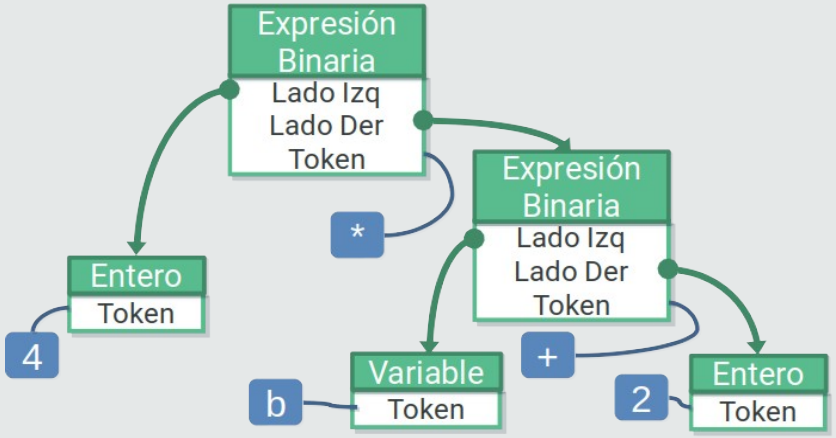
```
...
4*(b+2)
...
```



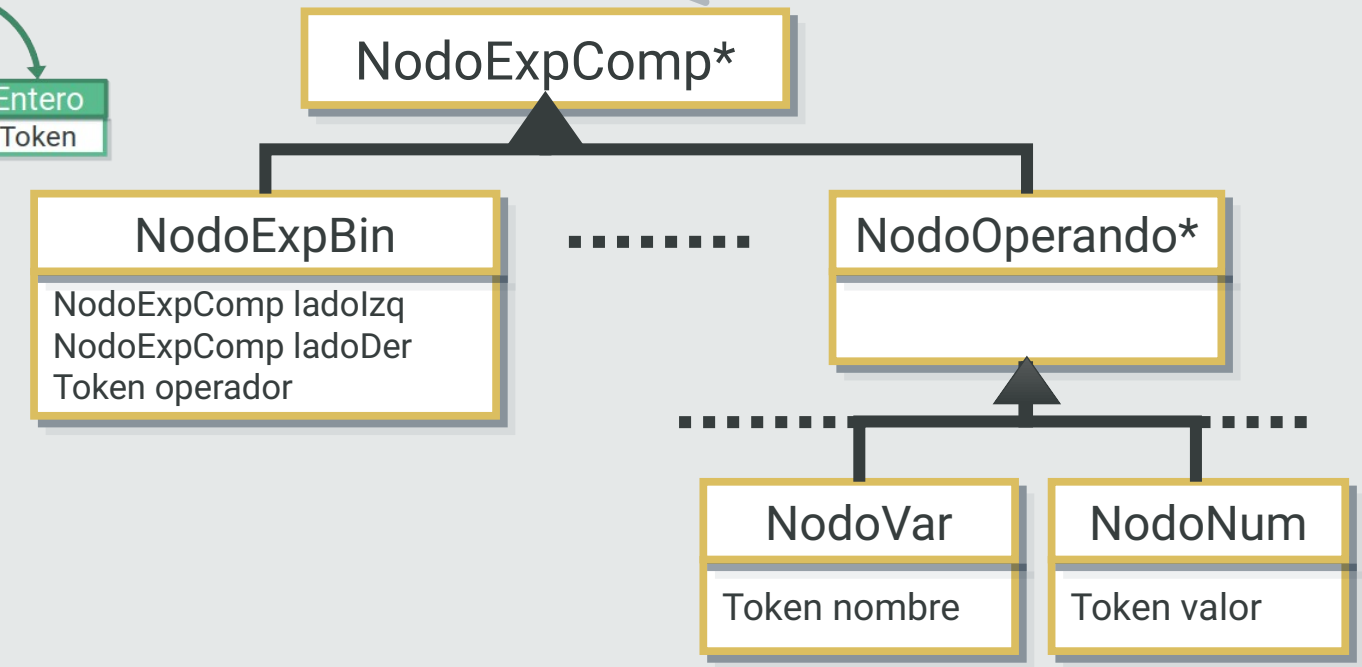
Ahí vimos que debería darnos este AST

Traduciendo Recurs

```
...
4*(b+2)
...
```

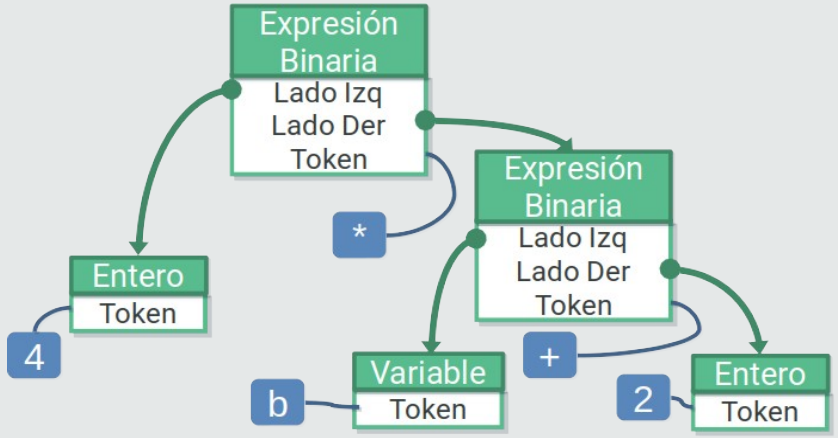


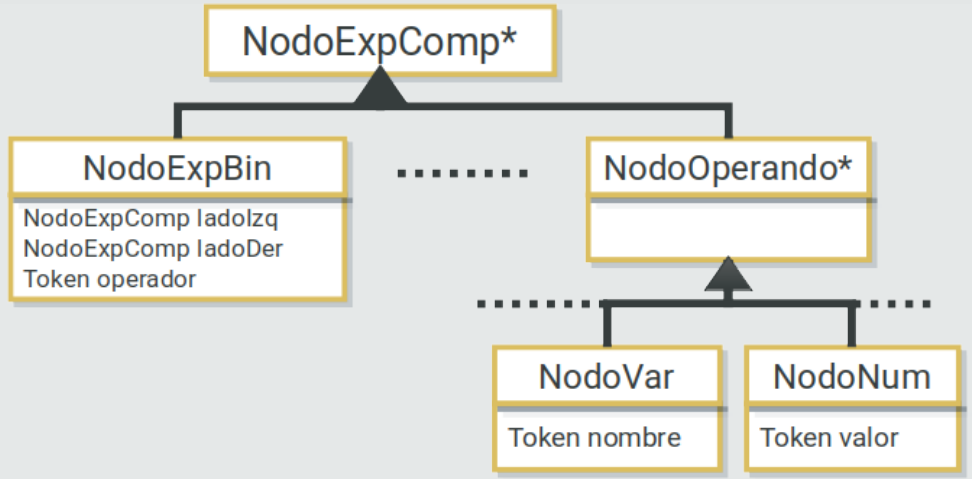
Por otro lado en 4.5.2 vimos como diseñar los nodos del AST y en base a los ejemplos podríamos tener estos nodos para la parte de expresiones



Traduciendo Recursion a Izquierda

```
...
4*(b+2)
...
```



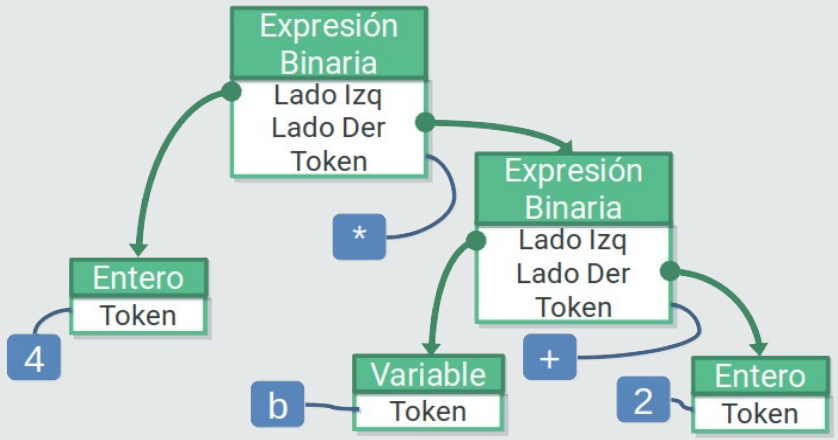


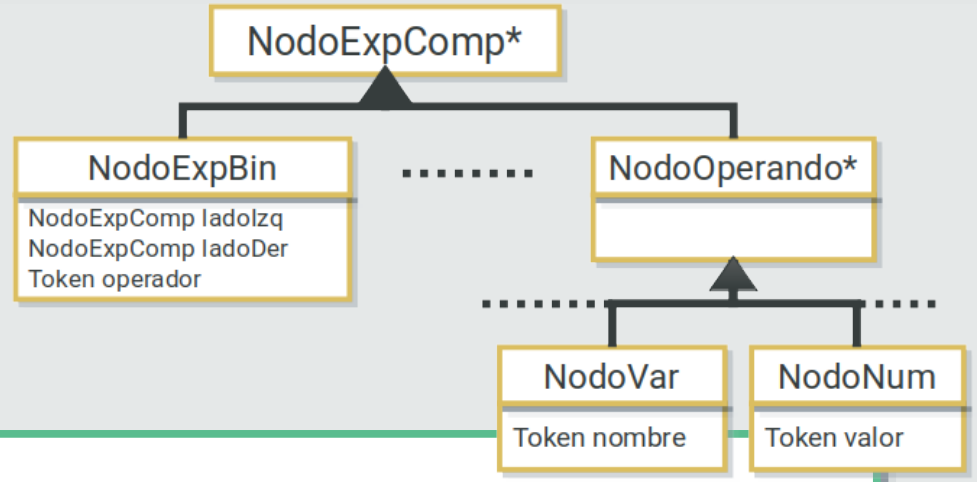
Exp → Operando Resto
Resto → OpBin Operando Resto
Resto → ε
OpBin → **+**
OpBin → ***
Operando → **litInt**
Operando → **id**
Operando → **(**Exp**)**

Supongamos que esta es la gramática que utilizamos para construir el analizador Sintáctico… ¿Que haríamos en el Parser para crear los ASTs de las expresiones en base al diseño de arriba?

Traduciendo Recursion a Izquierda

```
...
4*(b+2)
...
```





Exp → Operando Resto
Resto → OpBin Operando Resto
Resto → ε
OpBin → +
OpBin → *
Operando → litInt
Operando → id
Operando → (Exp)

Exp »
Operando Resto »
litInt **Resto** »
litInt **OpBin** Operando Resto »
litInt * **Operando** Resto »
litInt * (**Exp**) Resto »
litInt * (**Operando** Resto) Resto »
litInt * (id **Resto**) Resto »
litInt * (id + **Operando** Resto) Resto »
litInt * (id + litInt **Resto**) Resto »
litInt * (id + litInt) **Resto** »
litInt * (id + litInt)

Traduciendo Recursion a Izquierda

```
NodoExp Exp()
    NodoExp li = Operando()
    return Resto(li)

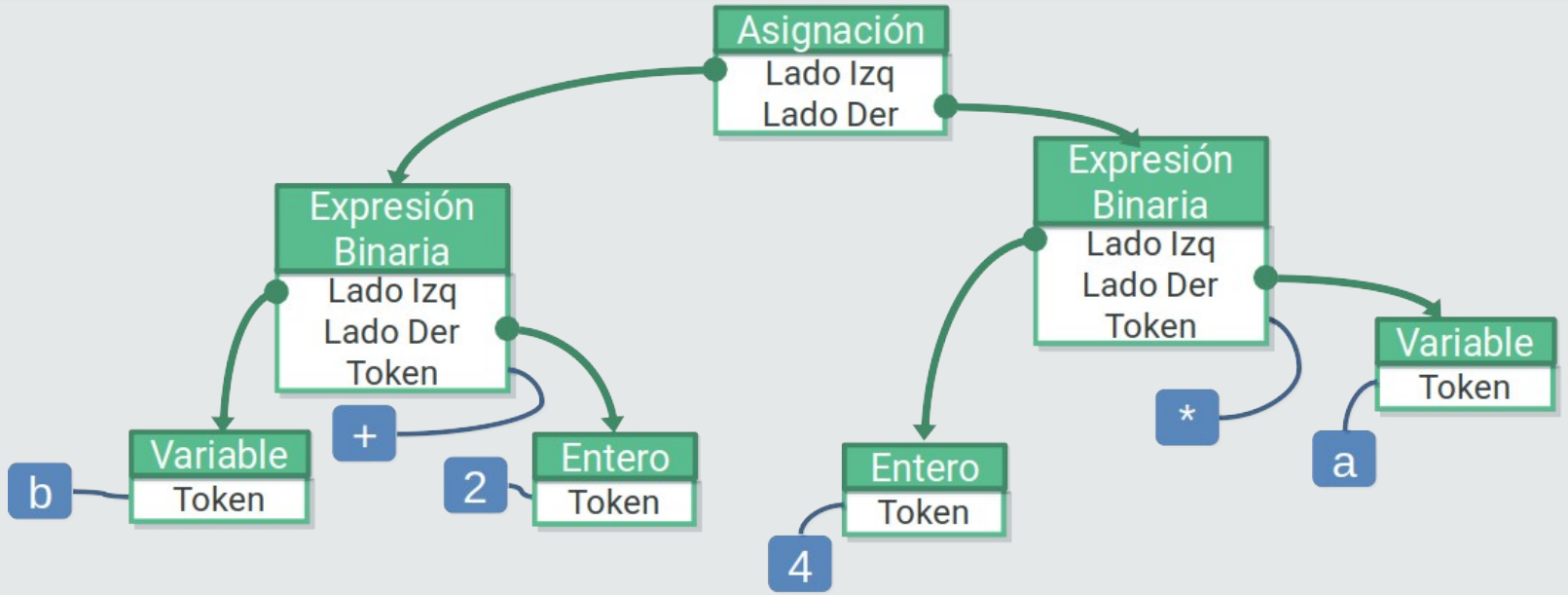
NodoExp Resto(NodoExp li)
    if(tkActual es "+" o "*")
        String op = OpBin()
        NodoExp ld = Operando()
        NodoBin nb = new NodoBin(op, li, ld)
        return Resto(nb)
    else
        return li
String OpBin()
    if(tkActual es "+")
        match("+")
        return "+"
    else if(tkActual es "*")
        match("*")
        return "*"
    else ERROR!
```

```
NodoExp Operando()
    if(tkActual es "litInt")
        NodoNum nn = new NodoNum(tkActual.lex)
        match("litInt")
        return nm
    else if(tkActual es "id")
        NodoVar nv = new NodoVar(tkActual.lex)
        match("id")
        return nv
    else if(tkActual es "(")
        match("(")
        NodoExp ne = Exp()
        match(")")
        return ne
    else ERROR!
```

Información Estructural

Chequeo Sentencias – ASTs – Algo mas?

```
…
b+2 = 4*a;
…
```



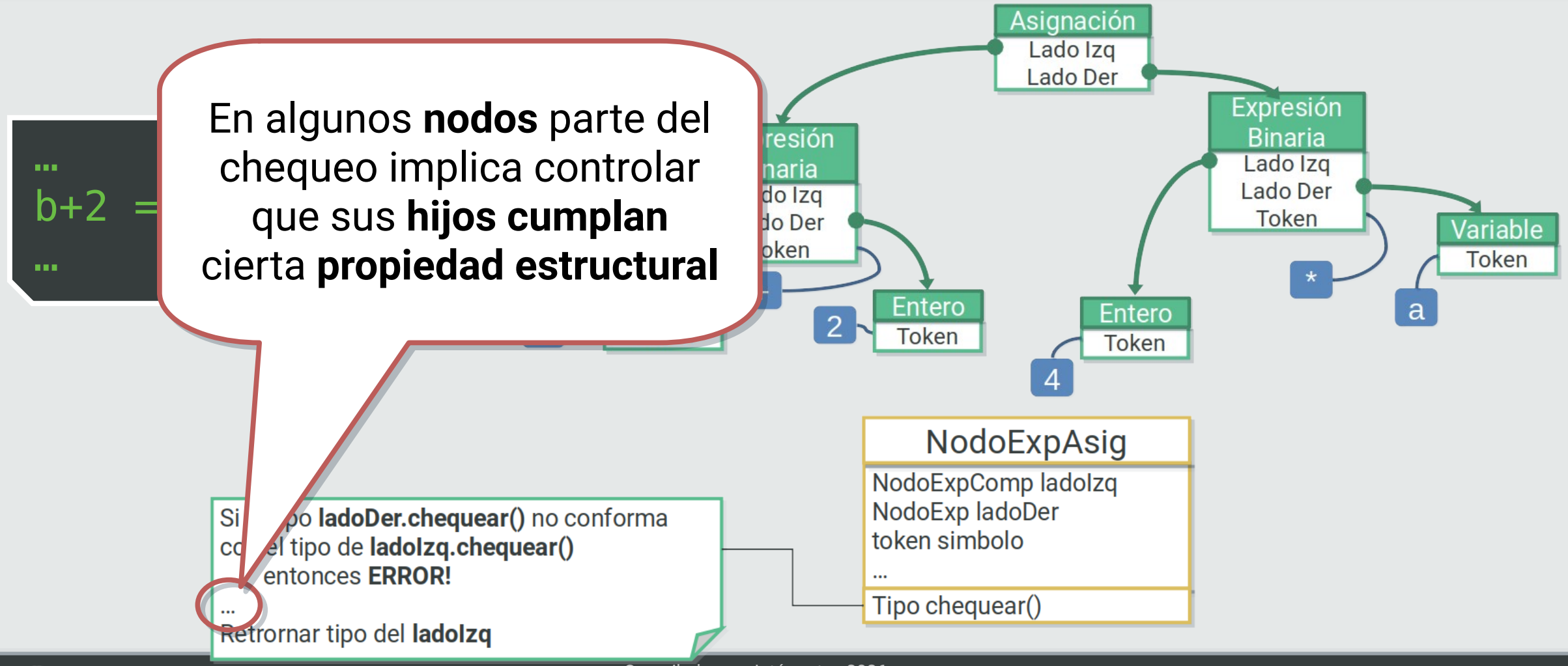
NodoExpAsig

NodoExpComp ladoIzq
NodoExp ladoDer
token simbolo
…

Tipo chequear()

Si el tipo **ladoDer.chequear()** no conforma con el tipo de **ladoIzq.chequear()**
entonces **ERROR!**
…
Retornar tipo del **ladoIzq**

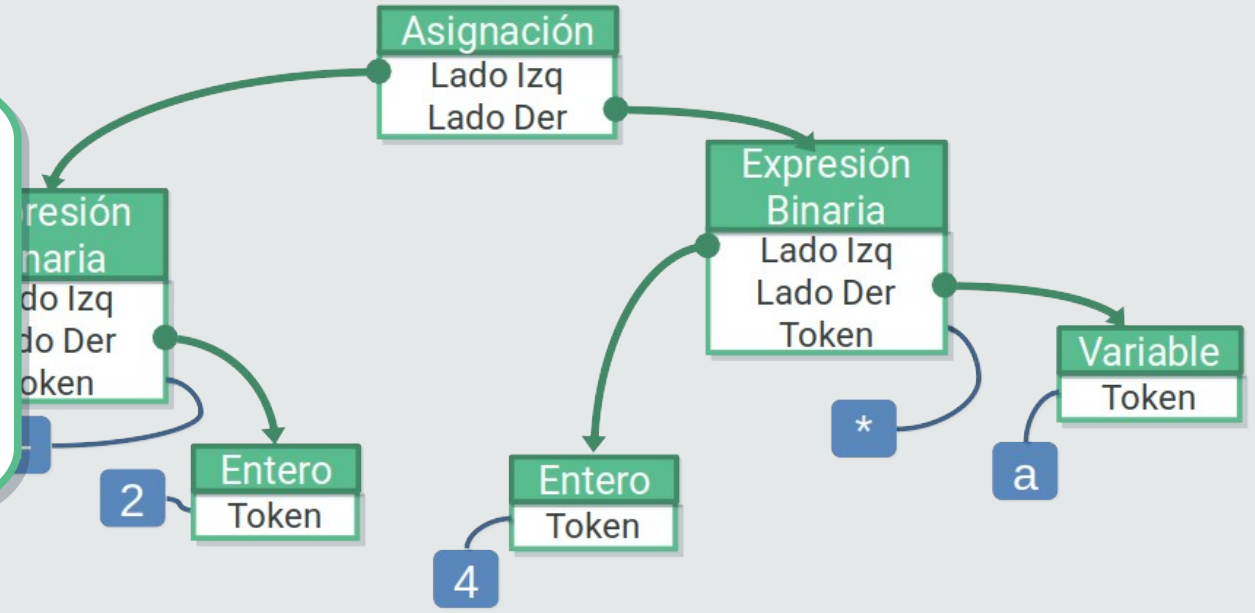
Chequeo Sentencias – ASTs – Algo mas?



Chequeo Sentencias – ASTs – Algo mas?

```
...
b+2 =
...
```

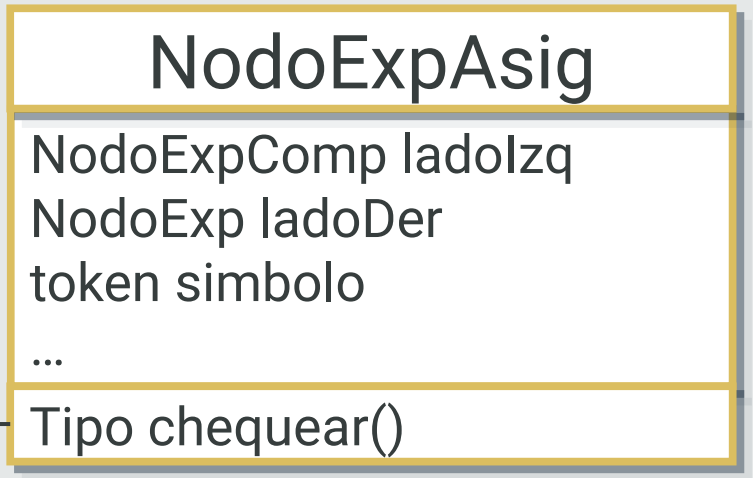
En una **asignación**, esperamos que el **lado izquierdo** sea algo a lo que se pueda asignar un valor… por ejemplo una variable



Si el tipo **ladoDer.chequear()** no conforma con el tipo de **ladoIzq.chequear()**
entonces **ERROR!**
...
Retornar tipo del **ladoIzq**

ASTs – Chequeo de Propiedad Estructural

Si el tipo **ladoDer.chequear()** no conforma
con el tipo de **ladoIzq.chequear()**
entonces **ERROR!**
...
Si **ladoIzq** NO es Asignable
entonces **ERROR!**
...
Retrornar tipo del **ladoIzq**



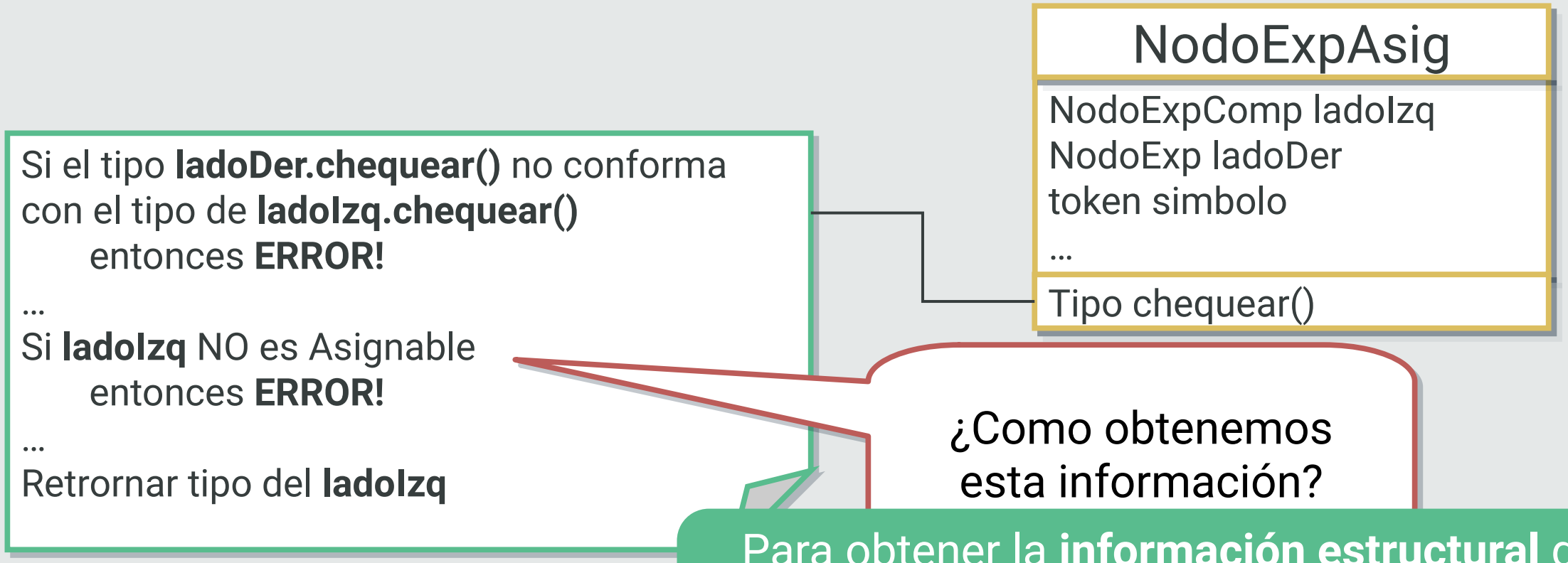
ASTs – Chequeo de Propiedad Estructural

Si el tipo **ladoDer.chequear()** no conforma con el tipo de **ladoIzq.chequear()**
entonces **ERROR!**
...
Si **ladoIzq** NO es Asignable
entonces **ERROR!**
...
Retrornar tipo del **ladoIzq**

NodoExpAsig
NodoExpComp ladoIzq NodoExp ladoDer token simbolo ...
Tipo chequear()

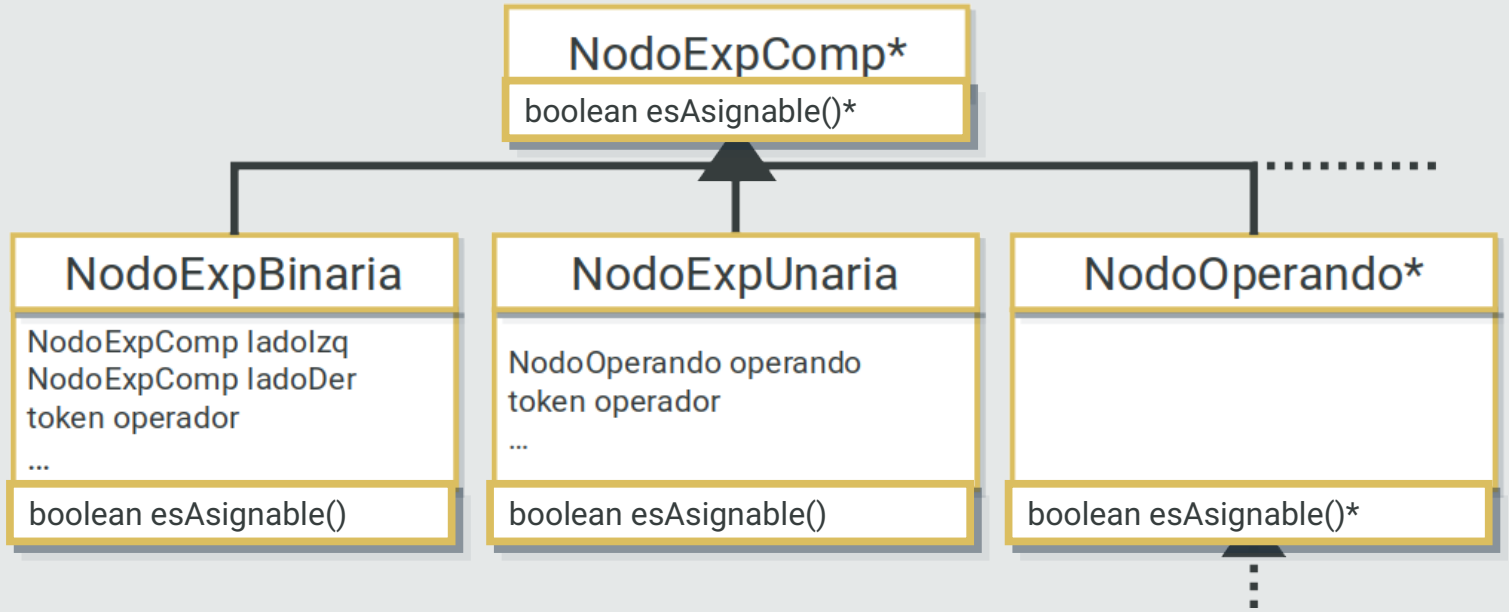
¿Como obtenemos esta información?

ASTs – Chequeo de Propiedad Estructural



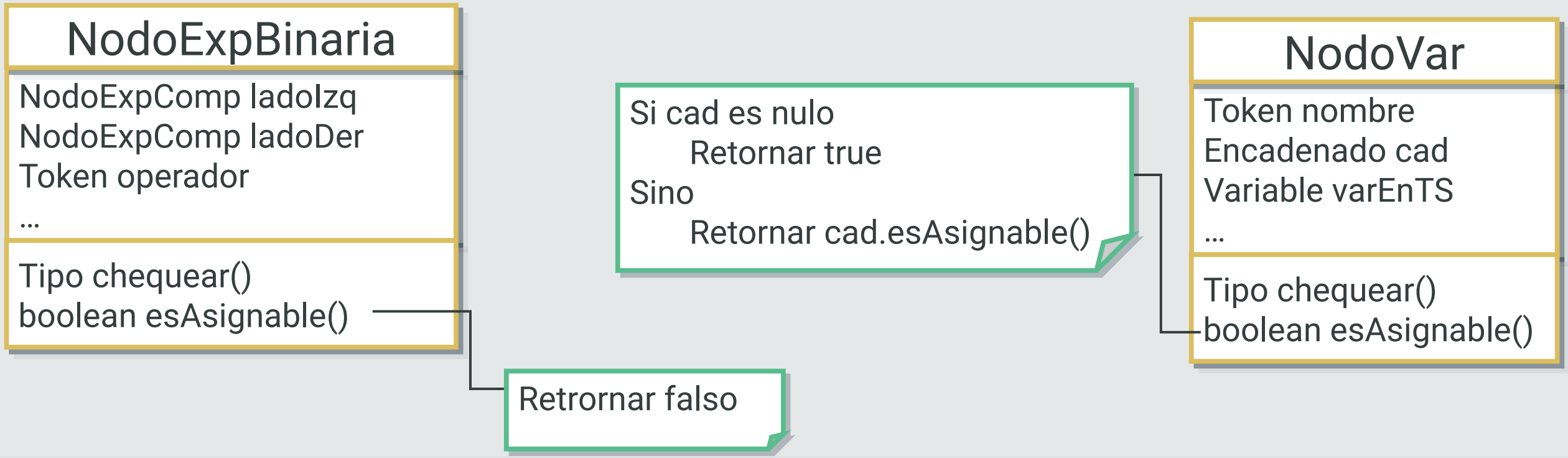
ASTs – Info Estructural – Alternativa 1

- Los nodos expresión cuentan un servicio que calculan si el nodo cumple esa propiedad



ASTs – Info Estructural – Alternativa 1

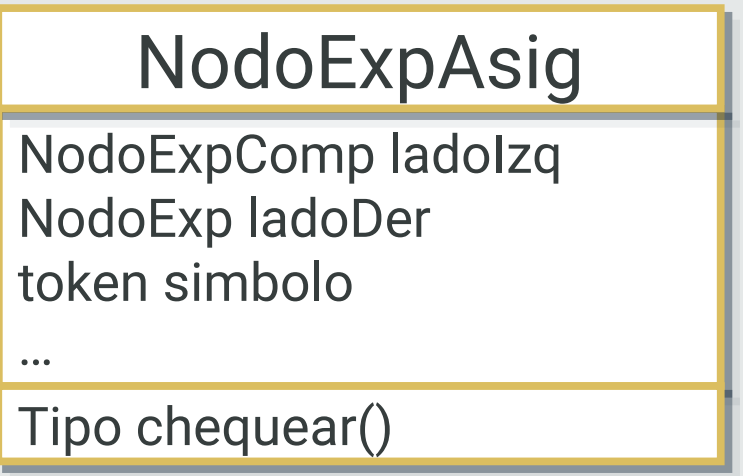
- Los nodos expresión cuentan un servicio que calculan si el nodo cumple esa propiedad



ASTs – Info Estructural – Alternativa 1

- Solicitar al nodo a partir de un servicio que calcule si cumple esa propiedad

Si el tipo **ladoDer.chequear()** no conforma con el tipo de **ladoIzq.chequear()**
entonces **ERROR!**
…
Si NO **ladoIzq.esAsignable()**
entonces **ERROR!**
…
Retrornar tipo del **ladoIzq**



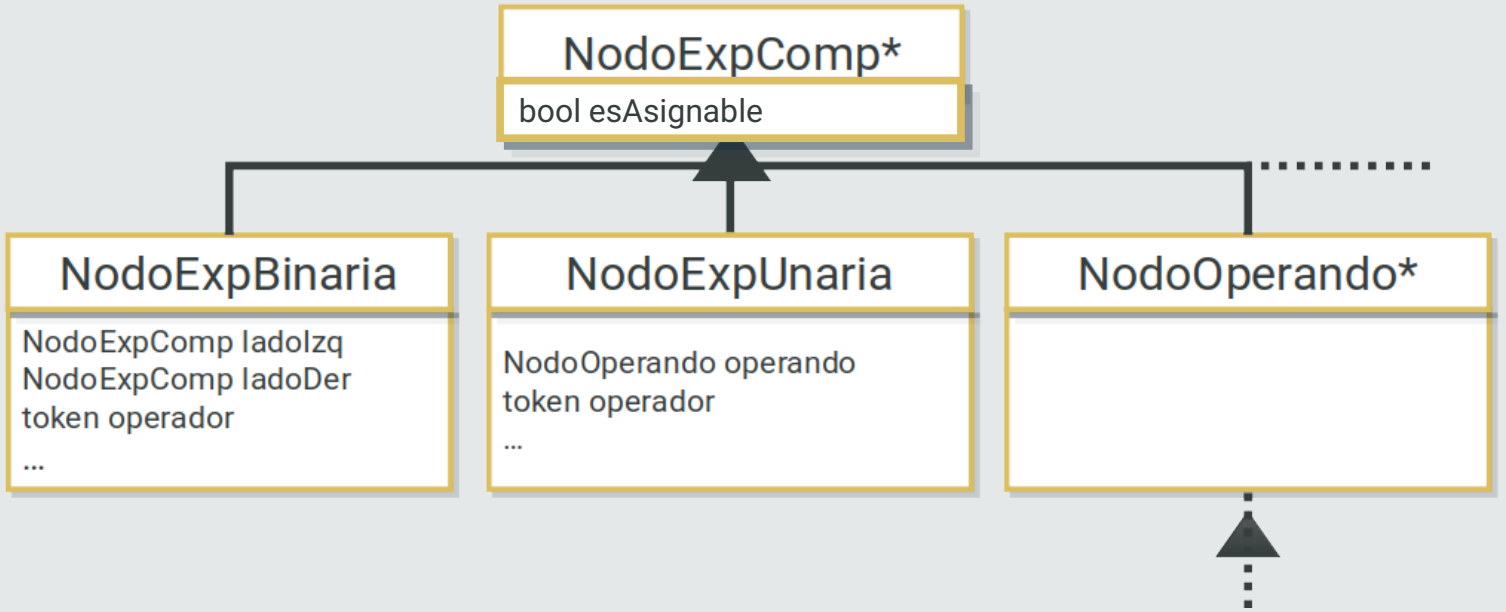
ASTs – Info Estructural – Alternativa 1

- Es la alternativa mas **sencilla** de implementar y utilizar
- Es **limpia** ya que se **desacopla** del **chequeo** de tipos y resolución de nombres

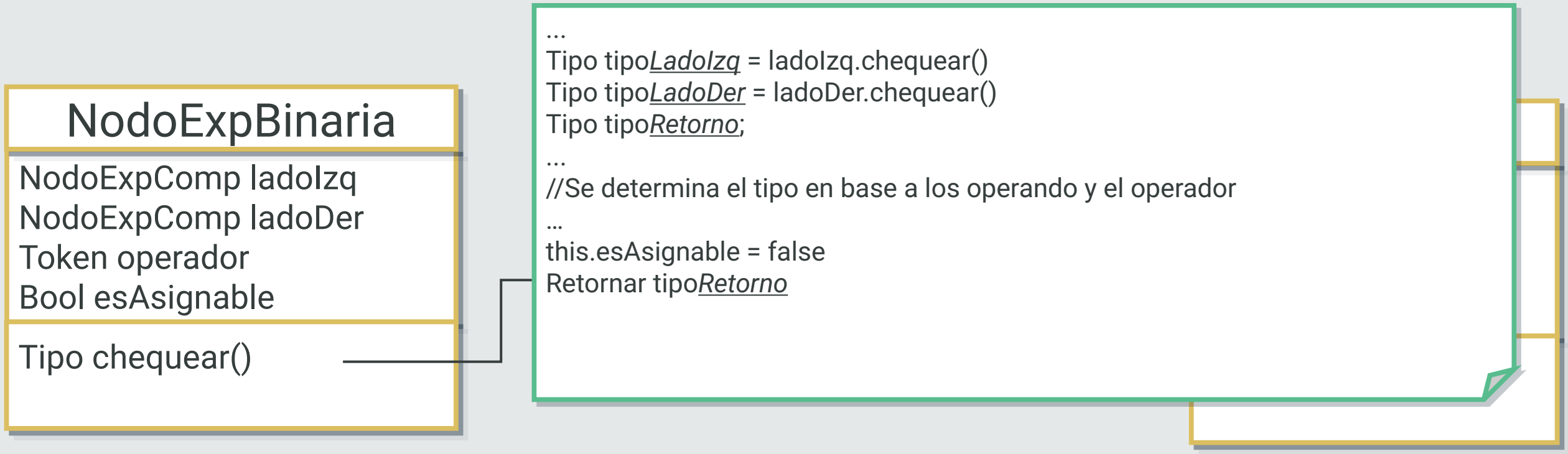
- Es **ineficiente** ya que requiere **volver a recorrer** los hijos (ya se recorrieron una vez con el chequear)
  - Mas aun, si hay **mas de un tipo** de información estructural habrá **múltiples recorridos** (uno por cada servicio)

ASTs – Info Estructural – Alternativa 2

- Todos los nodos expresion tienen una propiedad esAsignable que se determina en el servicio chequear del nodo



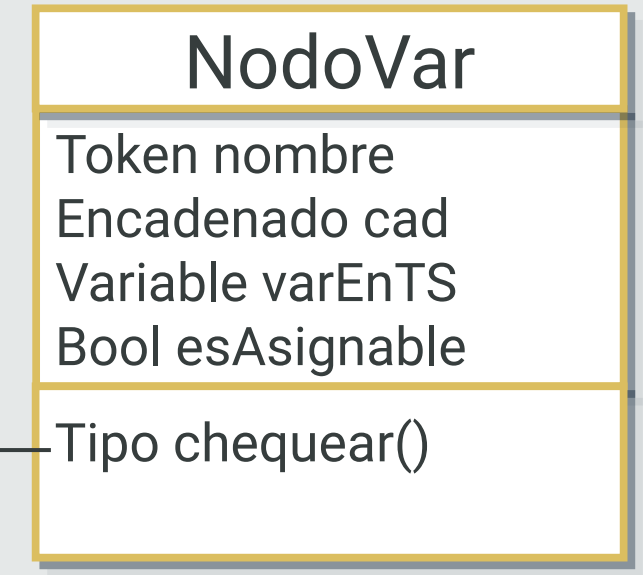
ASTs – Info Estructural – Alternativa 2



ASTs – Info Estructural – Alternativa 2

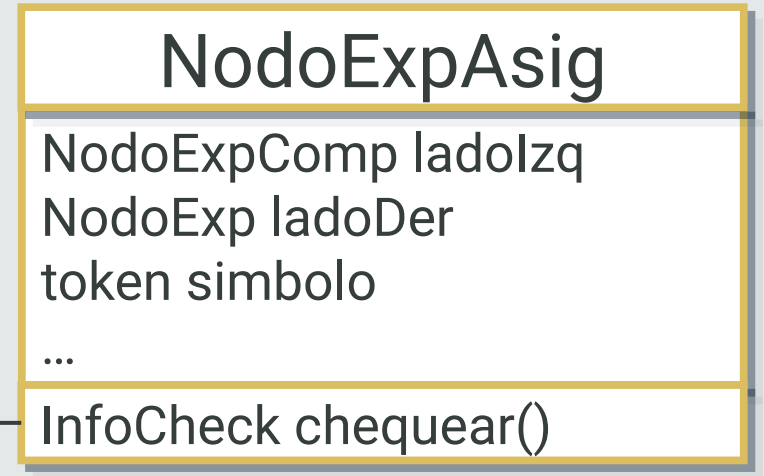
```
...
Si TS.metodoActual().esParametro(nombre)
        varEnTS = TS.metodoActual().getParametro(nombre)
Sino Si TS.bloqueActual().esVarLocal(nombre)
        varEnTS = TS.bloqueActual().getVarLocal(nombre)
Sino Si TS.claseActual().esAtributo(nombre) y TS.metodoActual().esDinamico()
        varEnTS = TS.claseActual().getAtributo(nombre)
Sino ERROR!

Tipo tipoRetorno
Si cad no es nulo
        tipoRetorno = cad.chequear(varEnTS.getTipo())
        this.esAsignable = cad.esAsignable;
sino
        tipoRetorno = varEnTS.getTipo()
        esAsignable = true
Retornar tipoRetorno
```



ASTs – Info Estructural – Alternativa 2

```
Tipo tipoLadoIzq = ladoIzq.chequear()
Tipo tipoLadoDer = ladoDer.chequear()
Tipo tipoRetorno = new InfoCheck()
Si el tipo tipoLadoDer conforma con el tipo de tipoLadoIzq
        tipoRetorno = tipoLadoDer
Sino ERROR!
...
Si NO ladoIzq.esAsignable
        entonces ERROR!
...
this.esAsignable = falso
Retrornar tipoRetorno
```



ASTs – Info Estructural – Alternativa 2

- Es mas eficiente:
  - No requiere recorrer el AST mas de una vez
  - Ahorra controles

- Es mas “sucia” y hace que el método chequear crezca mas en los nodos como comportamientos mas complejos (variables y llamadas)